

Aprendizado por Reforço em GPU para o Pêndulo-N: um Estudo de Throughput, Precisão Mista e Escalabilidade

Vitor Y. F. Freitas¹

¹Programa de Pós-Graduação em Engenharia Elétrica e de Computação (PPgEEC)
Universidade Federal do Rio Grande do Norte (UFRN) – Natal, RN – Brasil

vitoryeso@outlook.com

Abstract. *The swing-up of a cart-mounted N -link pendulum is a canonical underactuated, chaotic control problem whose difficulty grows sharply with N . We argue, and verify empirically, that policy capacity is not the bottleneck — the bottleneck is exploration, which is fought with samples. This paper is a study along three axes. Throughput: a fully GPU-resident, N -generic environment sustains 7–8 million environment steps per second end-to-end (environment + PPO) on a single RTX 3060. Mixed precision: *bf16* yields $1.87\times$ end-to-end without degrading learning, while kernel fusion and CUDA graphs yielded little — the real gains came from precision, not kernelization. Scalability: the throughput budget enables trillion-step experiments on consumer hardware; under a uniform 50-billion-step budget, swing-up is solved up to $n=3$ (94%/86% of the reward ceiling) — at $n=1$ also blind ($x, \dot{x}$ only, LSTM, 96.8%) — then collapses at $n=4$ ($\sim 20\%$), and $n=5$ remains open even after 1.6 trillion steps ($\sim 27\%$).*

Resumo. *O swing-up de um pêndulo de N elos montado em carrinho é um problema canônico de controle subatuado e caótico, cuja dificuldade cresce rapidamente com N . Argumentamos, e verificamos empiricamente, que a capacidade da política não é o gargalo — o gargalo é a exploração, combatida com amostras. Este artigo é um estudo em três eixos. Throughput: um ambiente genérico em N , residente 100% na GPU, sustenta 7–8 milhões de passos de ambiente por segundo ponta a ponta (ambiente + PPO) em uma única RTX 3060. Precisão mista: *bf16* rende $1,87\times$ ponta a ponta sem degradar o aprendizado, enquanto fusão de kernels e CUDA graphs renderam pouco — o ganho real veio da precisão, não da kernelização. Escalabilidade: o throughput viabiliza experimentos de trilhões de passos em hardware de consumo; sob um orçamento uniforme de 50 bilhões de passos, o swing-up é resolvido até $n=3$ (94%/86% do teto) — em $n=1$ inclusive às cegas (apenas $x, \dot{x}$, LSTM, 96,8%) —, despenca em $n=4$ ($\sim 20\%$) e permanece aberto em $n=5$ mesmo após 1,6 trilhão de passos ($\sim 27\%$).*

1. Introdução

Equilibrar um pêndulo invertido sobre um carrinho é um problema clássico de controle. Empilhar N elos rígidos e passivos sobre o mesmo carrinho o transforma em um dos problemas mais difíceis do controle não linear: o sistema passa a ter $n + 1$ graus de liberdade (carrinho mais n ângulos), mas continua com um único atuador, a força horizontal aplicada ao carrinho. Todas as juntas entre elos são passivas, movidas apenas por gravidade

e inércia. O sistema é *subatuado* e, para $n \geq 2$, caótico: pequenas diferenças de estado divergem exponencialmente. O *swing-up*, que consiste em partir da posição pendurada e levar todos os elos à vertical invertida, mantendo-os lá, é o cenário mais exigente desse benchmark [Boubaker 2013].

Este trabalho aborda o swing-up do pêndulo- N com aprendizado por reforço (RL) profundo, usando o algoritmo PPO (*Proximal Policy Optimization*) [Schulman et al. 2017], sob uma restrição adicional: a ação é discreta, com apenas três níveis ($\{-10, 0, +10\}$ N, *bang-off-bang*). Isso impede qualquer ajuste fino de controle contínuo; a política precisa aprender a ganhar energia por chaveamento, balançando o sistema.

A tese central deste artigo é uma questão de alocação de recursos: a capacidade da rede não é o gargalo. O estado do pêndulo- N é pequeno (a observação tem $2 + 3n$ dimensões; para $n=21$, apenas 65) e uma rede MLP (*multilayer perceptron*) pequena é suficiente para representar a política. O gargalo real é a exploração sob dinâmica instável e subatuada, e exploração se resolve com mais amostras. Por isso a alavanca científica deste trabalho é o throughput: cada otimização de sistemas que multiplica os passos por segundo multiplica também o orçamento de exploração disponível.

Diferente da abordagem usual, que busca arquiteturas maiores ou algoritmos de RL mais sofisticados, este trabalho investe em fazer *o mesmo* PPO processar ordens de grandeza mais experiência: de 500 bilhões a 1,6 trilhão de passos por experimento, em uma única GPU de consumo (RTX 3060, 12 GB) — viável porque o ambiente inteiro reside na GPU e o laço de treino foi otimizado e *medido* camada por camada.

O estudo se organiza em três pilares, que dão nome ao artigo e estruturam as Seções 4 e 5: (i) **throughput**, o ambiente GPU-nativo que financia o orçamento de exploração; (ii) **precisão mista**, onde os ganhos reais de velocidade vieram; e (iii) **escalabilidade**, em dois sentidos que fazemos questão de distinguir — a do *sistema*, que foi entregue, e a do *problema* (crescer n), que permanece parcialmente aberta.

2. Estado da Arte

RL massivamente paralelo em GPU. A constatação de que o gargalo do RL profundo costuma ser a geração de experiência, não o gradiente, produziu uma linhagem de simuladores GPU-nativos. Isaac Gym [Makoviychuk et al. 2021] moveu física e treino para a GPU, eliminando o tráfego CPU-GPU e atingindo dezenas de milhares de ambientes simultâneos. Brax [Freeman et al. 2021] implementou física diferenciável em JAX com a mesma filosofia; EnvPool [Weng et al. 2022] e Madrona [Shacklett et al. 2023] generalizaram o padrão de execução paralela de ambientes, reportando milhões de passos por segundo. No lado do algoritmo, Sample Factory [Petrenko et al. 2020] otimizou PPO assíncrono para 10^5 passos/s em uma máquina, e PufferLib [Suárez 2024], biblioteca sobre a qual este trabalho se apoia, padronizou a vetorização de ambientes nativos de alta velocidade. Esses trabalhos constroem simuladores de propósito geral e demonstram throughput em dezenas de tarefas. Nosso trabalho segue a mesma linhagem, mas com foco invertido: um único problema de controle clássico e uma única GPU de consumo, usados como cenário controlado para uma pergunta que esses trabalhos não isolam — aumentar o throughput de treino se traduz em *quanto* ganho de desempenho de controle?

Algoritmo. Usamos PPO [Schulman et al. 2017] com GAE (*Generalized Advan-*

tage Estimation) [Schulman et al. 2016], a combinação dominante em RL *on-policy* pela estabilidade do objetivo com razão de importância recortada (*clipped surrogate*) e pelo controle de viés–variância do GAE.

Controle subatuado e swing-up. O swing-up de sistemas subatuados tem tradição própria na teoria de controle: Spong [Spong 1995] formalizou o Acrobot e as estratégias de colocação parcial de energia; Åström e Furuta [Åström e Furuta 2000] derivaram o swing-up por modelagem de energia (*energy-based control*), que permanece a referência analítica para $n=1$. Para $n \geq 2$ não há solução analítica geral; abordagens clássicas combinam uma fase de balanço, para ganho de energia, com um regulador linear na vizinhança do topo. Nossa escada de n (Seção 3.2), com elos idênticos e apenas n variando, isola o efeito do número de elos sobre a dificuldade do swing-up.

Precisão reduzida. O treino em precisão mista [Micikevicius et al. 2018] estabeleceu que redes toleram aritmética de 16 bits com acumulação em 32; nós verificamos que isso vale também para o laço completo de PPO (política e ambiente) em bf16 (bfloat16).

3. Metodologia

3.1. Formulação do ambiente: o pêndulo- N sobre carrinho

O sistema é um carrinho sobre trilho limitado ($|x| \leq 5$ m) com n elos rígidos idênticos acoplados em série; apenas o carrinho é atuado, e todas as juntas entre elos são passivas. A dinâmica segue da formulação lagrangiana. A cada passo, monta-se um sistema linear $A\ddot{q} = b$, em que A é a matriz de massa $(n+1) \times (n+1)$ do conjunto carrinho-mais-elos e b reúne as forças de gravidade, os termos inerciais (centrífugos e de Coriolis) e a força aplicada ao carrinho. Esse sistema é resolvido por eliminação gaussiana a cada passo e por ambiente, e o estado é integrado com passo dt (convenção: $\theta=0$ é a vertical *invertida*). A estrutura de A é regular em n : sendo m e l a massa e o comprimento de cada elo e θ_k o ângulo do elo k ,

$$\begin{aligned} A_{00} &= M_c + n m, & A_{0k} &= (n-k+1) m l \cos \theta_k, \\ A_{jk} &= (n - \max(j, k) + 1) m l^2 \cos(\theta_j - \theta_k), \end{aligned} \quad (1)$$

com $j, k \geq 1$ (a diagonal A_{jj} é o caso $j=k$). Em cada termo, $(n-k+1) m$ é a massa efetiva carregada pela junta k — todos os elos acima dela — e o acoplamento entre elos depende do cosseno da diferença de ângulos. Essa regularidade permite montar e resolver a dinâmica com o mesmo código para qualquer número de elos (realização em GPU: Seção 3.5). A Tabela 1 lista os parâmetros físicos usados, e a Figura 1 ilustra o ambiente e o comportamento aprendido.

Tabela 1. Parâmetros físicos do ambiente pêndulo- N .

Massa do carrinho M_c	1,0 kg	Gravidade g	9,8 m/s ²
Massa por elo m	0,1 kg	Passo de integração dt	0,02 s
Comprimento por elo l	0,35 m	Passos por episódio	1500
Força aplicada	± 10 N (3 níveis)	Limite do trilho	$ x \leq 5$ m

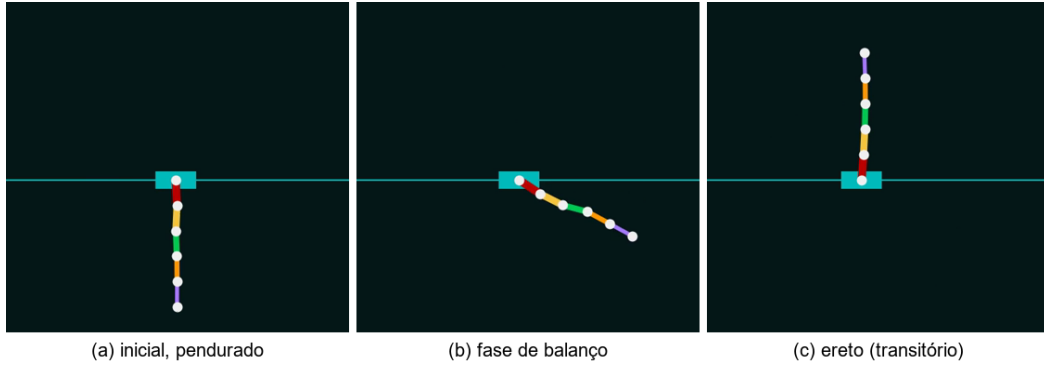


Figura 1. Episódio do agente treinado em $n=5$: (a) estado inicial pendurado; (b) fase de balanço (ganho de energia por chaveamento); (c) os cinco elos eretos, de forma *transitória* — atingir o topo é aprendido; *manter* é a parte não resolvida (Seção 4.3).

Observação e ação. A observação normalizada tem $2 + 3n$ dimensões:

$$o = [x/5, \dot{x}/5, (\sin \theta_k, \cos \theta_k, \dot{\theta}_k/8)_{k=1}^n], \quad (2)$$

com $\sin / \cos$ no lugar do ângulo cru para evitar a descontinuidade circular em $\theta = \pm\pi$. A ação é discreta com três níveis, $a \in \{-10, 0, +10\}$ N (*bang-off-bang*); o reset posiciona o sistema próximo da configuração pendurada, com ruído.

Recompensa. A recompensa por passo da linha principal é

$$r = \text{clamp}((0,5 \cdot \text{height} + \text{hold}) \cdot \text{center}, 0, 1,5), \quad (3)$$

onde height mede a elevação média dos elos, hold bonifica permanência próxima ao topo e $\text{center} = 1 - (x/x_{\max})^2$ penaliza afastamento do centro do trilho. O teto por episódio é ~ 2250 . Importante: o fator multiplicativo center é *farmável* — um agente pode colher recompensa parado no centro sem levantar o pêndulo. Esse defeito foi detectado e corrigido na variante POMDP (Seção 3.4); na linha principal em CUDA ele permanece, o que é uma limitação explícita deste trabalho (Seção 5.5).

3.2. A escada de n : dificuldade crescente

A dificuldade é parametrizada pelo número de elos n . Cada elo mantém massa e comprimento fixos (Tabela 1), de modo que a escada $n = 1, 2, 3, 5, 8, 13, 21$ acrescenta graus de liberdade e acoplamento a uma cadeia progressivamente mais longa. Ela percorre regimes de dificuldade crescente, de um corpo quase rígido ($n \leq 3$) a uma cadeia articulada fortemente caótica ($n \geq 8$), em que o acoplamento entre elos endurece o controle. O comprimento de 0,35 m por elo é compatível com bancadas didáticas reais da linha Quanser (pêndulos de 33,65 e 64,13 cm). Este artigo reporta resultados até $n = 5$; a escada completa é o programa de pesquisa.

3.3. Algoritmo: PPO com GAE

O treino usa o PPO [Schulman et al. 2017], um método de RL *on-policy*: a política corrente coleta um lote de transições e é então atualizada por alguns passos de gradiente sobre esse lote, maximizando o objetivo *surrogate* recortado. Seja $\rho_t = \pi_\theta(a_t | s_t) / \pi_{\theta_{\text{old}}}(a_t | s_t)$

a *razão de importância*: $\rho_t < 1$ indica que a ação ficou menos provável na política nova; $\rho_t > 1$, mais provável. O objetivo é

$$L(\theta) = \mathbb{E}_t \left[\min(\rho_t \hat{A}_t, \text{clip}(\rho_t, 1-\epsilon, 1+\epsilon) \hat{A}_t) \right]. \quad (4)$$

Aqui $\hat{A}_t$ é a *vantagem*: quão melhor a ação foi do que a média prevista pelo crítico V — subtrair essa média centra o sinal de aprendizado em torno de zero e reduz a variância do gradiente. O recorte $\epsilon=0,2$ impede que um único *update* mova a política para longe demais, o que dá estabilidade ao PPO; é ele que torna seguro reutilizar o mesmo lote em vários minibatches e épocas, prática incomum em RL *on-policy* [Barth 2026]. A vantagem é estimada por **GAE** [Schulman et al. 2016]: uma soma dos erros temporais $\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$ ponderada por $(\gamma\lambda)$, em que γ é o fator de desconto (horizonte efetivo) e λ ajusta o compromisso viés–variância da estimativa. Ainda na Tabela 2: o *coeficiente de valor* pesa a perda do crítico; o *coeficiente de entropia* bonifica políticas estocásticas, preservando exploração; norma máxima de gradiente e *clip* de valor são estabilizadores numéricos. A política é uma MLP: codificador linear $\rightarrow$ 2 camadas ocultas de 256 unidades $\rightarrow$ decodificador com cabeças de ação (3 logits) e de valor. A Tabela 2 resume a configuração de $n=5$; os demais n variam apenas a escala de paralelismo.

Tabela 2. Hiperparâmetros do PPO (configuração $n=5$).

Ambientes paralelos	32 768	$\gamma / \lambda_{\text{GAE}}$	0,99 / 0,95
Horizonte de rollout	128	Clip (política / valor)	0,2 / ± 1
Minibatches / épocas	16 / 1	Coef. de valor	0,5
Otimizador	Adam	Norma máx. de gradiente	0,5
LR (cosseno)	$1,5 \times 10^{-4} \rightarrow 10^{-5}$	Entropia (anelada)	0,005 $\rightarrow$ 0,001
Política	MLP 2×256	Ação	discreta (3)

Para destravar platôs em orçamentos longos usamos *warm-restarts* da taxa de aprendizado (LR) ao estilo SGDR (*stochastic gradient descent with warm restarts*): a cada extensão de orçamento (500B $\rightarrow$ 1T $\rightarrow$ 2T), o LR retorna a $1,5 \times 10^{-4}$ e re-anela.

3.4. Variante POMDP: swing-up às cegas

Para isolar o papel da memória, construímos uma variante $n=1$ parcialmente observável (POMDP, *Partially Observable Markov Decision Process*): a política vê *apenas* $(x, \dot{x})$ — posição e velocidade do carrinho — e precisa inferir o estado do pêndulo a partir do histórico de reações do carrinho. O desenho experimental tem três braços: (i) observação completa + MLP (teto de referência); (ii) observação parcial + MLP sem memória (**controle negativo**: deve falhar, pois $(x, \dot{x})$ instantâneo é ambíguo); (iii) observação parcial + política recorrente LSTM (*Long Short-Term Memory*), a hipótese sob teste. Nessa variante a recompensa foi *de-confundida*: removemos o termo center farmável e medimos sucesso por métricas físicas lidas do estado real do simulador — $\cos \theta$ médio e %up (fração do tempo com $\cos \theta > 0,6$) — nunca pelo retorno proxy. Um refinamento essencial foi o TBPTT *stateful* (*truncated backpropagation through time* com estado persistente): cada janela de 128 passos do update é inicializada com o estado oculto real capturado no rollout, e não com $h=0$ (análise na Seção 5.4).

3.5. Implementação: ambiente GPU-nativo e pilha de execução

Ambiente GPU-nativo. As seções anteriores descreveram o método. Esta seção descreve como ele é implementado em GPU. A dinâmica da Seção 3.1 é um CuPy `RawKernel` (CUDA C compilado via NVRTC em tempo de execução), genérico em N por macro de compilação (`#define NUM_LINKS`): o mesmo código-fonte gera executáveis especializados para cada n , com laços de montagem da matriz e da eliminação gaussiana desenrolados pelo compilador, dentro do próprio *kernel*. Todos os buffers (estados, observações, recompensas, terminações) vivem na GPU; a fronteira com o PyTorch é *zero-copy* via DLPack — nenhum byte cruza o barramento PCIe durante o treino. A paridade numérica do *kernel* genérico foi validada contra uma implementação $n=4$ desenrolada à mão (diferença máxima $8,8 \times 10^{-7}$, ruído de fp32) e contra referência NumPy ($\sim 5 \times 10^{-7}$). O laço de PPO da Seção 3.3 é implementado à mão sobre a PufferLib [Suárez 2024], uma biblioteca de RL que padroniza a vetorização de ambientes de alta velocidade. É sobre a sua infraestrutura de vetorização e seus modelos (`pufferlib.models`) que o ambiente e o laço de treino deste trabalho foram construídos.

Pilha de execução. Sobre esse ambiente, quatro alavancas foram avaliadas — sempre com o *workload* (modelo, batch, ambiente) fixo, variando apenas a execução:

- **bf16 autocast:** rollout e update em `bfloat16` com acumulação fp32 (as camadas recorrentes da variante POMDP permanecem em fp32 por incompatibilidade de dtypes no estado persistente).
- **torch.compile:** compilação do grafo do update (no rollout, a compilação *piora* o tempo e foi desativada).
- **Kernels Triton fundidos (*fused*):** $\text{GELU}(XW + b)$ em um único *kernel*, eliminando uma ida e volta à HBM (*High Bandwidth Memory*, a memória principal da GPU) em relação a cuBLAS + GELU separados.
- **CUDA graphs:** captura do laço de rollout em grafo para eliminar overhead de lançamento de *kernels*.

3.6. Metodologia de medição

Três princípios disciplinam as medições, todos aprendidos por violação: **(1) ponta a ponta contra a produção real** — speedup medido no laço completo de treino com política *treinada*, nunca contra réplicas sintéticas (o porquê fica claro na Seção 5.2); **(2) roofline para atribuir causa** — localizar o laço no *roofline* da RTX 3060 (banda ~ 360 GB/s, pico ~ 51 TFLOP/s fp16/bf16, *ridge point* AI = 142 FLOP/byte) antes de creditar ganho a compute ou banda; e **(3) A/B de aprendizado, multi-semente e sustentado** — nenhuma otimização numérica entra sem A/B de *qualidade* (não só de velocidade), pois um *snapshot* curto pode mentir (Seção 5.2).

4. Resultados

Os resultados seguem os três pilares do título: throughput (Seção 4.1), precisão mista (Seção 4.2) e escalabilidade (Seção 4.3).

4.1. Pilar 1 — Throughput

O ambiente isolado (sem política) sustenta centenas de milhões de passos de ambiente por segundo (SPS): 333 M em $n=4$ e 260 M em $n=5$ — apenas $\sim 22\%$ mais lento apesar

de resolver um sistema 6×6 em vez de 5×5 , pois a eliminação gaussiana desenrolada em registradores escala suavemente. O laço completo (ambiente + PPO), após todas as otimizações, sustenta 7–8 milhões de passos/s (7,2 M em $n=4$; 7,7 M em $n=5$); a diferença é o custo do PPO (forward, GAE, backward), que domina o tempo de época. Para contexto: a linha legada do projeto, um ambiente distinto implementado em C sobre a CPU (*ctypes*) na mesma máquina, sustentava 125 K SPS no laço de treino; a reimplementação como *kernel* CUDA residente na GPU multiplicou o throughput do laço completo por $\sim 60\times$ (comparação laço contra laço, ambos ambiente + PPO). A Tabela 3 resume essa evolução, das referências históricas às otimizações de execução do Pilar 2.

Tabela 3. Evolução do throughput na RTX 3060, em dois blocos com referências próprias. SPS medido pelo próprio laço em janelas de 100 M passos; as duas primeiras linhas do segundo bloco convertem tempos de época medidos (4 194 304 passos/época) e a última é a média $\pm$ desvio das 2 000 janelas finais do treino de $n=5$ (inclui avaliação e checkpoints). TF32 = TensorFloat-32, o modo padrão de fp32 em *tensor cores*.

Etapa	SPS	Ganho (vs. etapa anterior)
<i>Implementação do ambiente (laço de treino completo)</i>		
Python/NumPy (<code>np.linalg.solve</code>)	$\sim 9,9$ K	$1\times$ (ref.)
→ C/CPU (<i>ctypes</i>)	~ 125 K	$\sim 13\times$
→ <i>kernel</i> CUDA (TF32)	$\sim 4,1$ M	$\sim 33\times$
<i>Otimizações de execução (ambiente CUDA + PPO)</i>		
TF32	$\sim 4,2$ M	$1\times$ (ref.)
+ bf16 autocast	$\sim 7,9$ M	$1,87\times$
+ <code>torch.compile</code>	$7,72 \pm 0,19$ M	$1,09\times$

Onde o laço vive no roofline. O GEMM (*General Matrix Multiplication*, a multiplicação de matrizes) dominante do update ($K=N=256$, a largura da MLP) tem intensidade aritmética $AI \approx 85$ FLOP/byte — abaixo do *ridge point* de 142 da RTX 3060. O laço é, portanto, **limitado por banda de memória**: o teto efetivo é $AI \times BW = 85 \times 360 \approx 30$ TFLOP/s, não os 51 teóricos de compute. A varredura do tamanho do GEMM (Figura 2) confirma: a eficiência cresce de 15,0 até saturar em $\sim 24,4$ TFLOP/s (**79% da banda**); no tamanho de produção ($M \approx 262\,144$, o M do update real) opera a 23,9 TFLOP/s, ou **78% da banda de memória**. Esse diagnóstico é a chave de leitura do pilar seguinte: se o custo dominante é tráfego de HBM, reduzir bytes (precisão) deve valer mais do que reduzir lançamentos de *kernel* (fusão, grafos). Foi exatamente o que se mediu.

4.2. Pilar 2 — Precisão mista

A Tabela 4 decompõe a contribuição de cada alavanca de execução avaliada, com veredito honesto.

bf16: a alavanca real. O bf16 é o resultado central deste pilar: $1,87\times$ ponta a ponta (época de 992 ms em TF32 para 529 ms em bf16), validado por A/B de *aprendizado* com 3 sementes a partir de um checkpoint de produção (+200M passos cada braço): retorno TF32 627,5 (desvio 27,3) vs. bf16 639,5 — diferença dentro do ruído — e zero NaN em 270 bilhões de passos subsequentes de produção. O roofline da Seção 4.1 explica

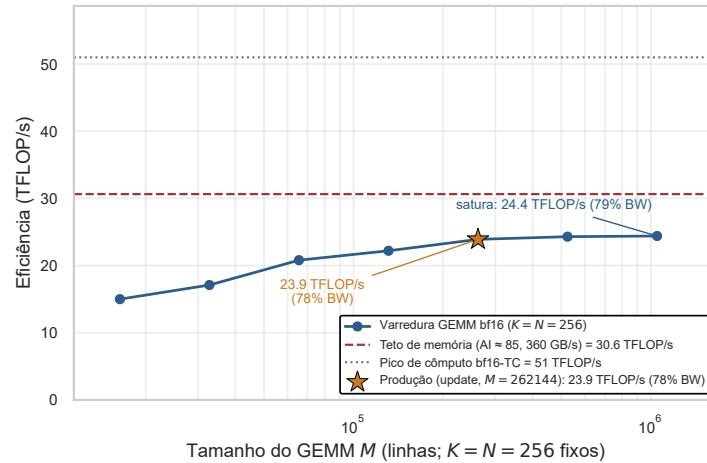


Figura 2. Varredura roofline do GEMM do update (bf16, $K=N=256$) na RTX 3060: no tamanho de produção ($M \approx 262\,144$) opera a 23,9 TFLOP/s (78% da banda), saturando em $\sim 24,4$ (79%) para M maiores — confirmando o regime memory-bound ($AI=85 < \text{ridge}=142$).

Tabela 4. Alavancas de execução e seu efeito ponta a ponta no laço de treino (microbenchmarks e vereditos discutidos no texto).

Alavanca	Efeito ponta a ponta (e2e)
bf16 autocast	$1,87 \times$ (992 $\rightarrow$ 529 ms)
<code>torch.compile</code> (update)	$1,09 \times$
Triton fundido (<i>fused</i>)	$1,02 \times$
CUDA graphs	$\sim 1,05 \times$ (paridade)
Staleness ($K=2$, revertida)	$1,35 \times$

o porquê: metade dos bytes por elemento $\Rightarrow$ o dobro de elementos pela mesma banda. Uma ablação com *workload* fixo (precisão $\times$ compilação) confirma que o eixo de precisão domina os ganhos, com a compilação contribuindo apenas marginalmente.

Kernelização: paridade e espantinhos. Pelo mesmo diagnóstico, as alavancas que não reduzem bytes renderam pouco. CUDA graphs deram $\sim 1,05 \times$ contra a produção real (o $1,52 \times$ inicial era medido contra um baseline espantinho; Seção 5.2). O *kernel* Triton fundido $\text{GELU}(XW+b)$ é legitimamente $2,88 \times$ mais rápido que cuBLAS + GELU separados em microbenchmark ($0,807 \rightarrow 0,281$ ms) — fusão elimina uma ida e volta à HBM em uma operação memory-bound — mas sobreviveu à integração como apenas $+2\%$ ponta a ponta, porque o rollout é só $\sim 30\%$ da época; a variante multi-camada totalmente fundida esbarrou no limite de memória compartilhada e não foi concluída. Em termos quantitativos, neste regime o ganho veio da redução de precisão, e não da fusão de *kernels*.

4.3. Pilar 3 — Escalabilidade

Distinguimos duas dimensões de escala, deliberadamente: escalar o *sistema* e escalar o *problema*. A primeira foi entregue; a segunda é onde o estudo encontra sua fronteira.

(a) **Escalar o sistema.** Os dois pilares anteriores compõem: 32 768 ambientes

paralelos, 7–8M passos/s sustentados e estabilidade numérica (zero NaN sob bf16) para campanhas ininterruptas de dias, tornando executáveis, em uma GPU de consumo, orçamentos de 10^{11} – 10^{12} passos — o experimento $n=5$ acumulou 1,6 trilhão.

(b) Escalar o problema: a escada de n .

A Tabela 5 consolida o melhor resultado de controle por n (orçamentos indicados) e a Figura 3 isola o efeito de n comparando todos os tamanhos sob o *mesmo* orçamento de 50 B passos. O teto teórico de retorno por episódio é ~ 2250 (1500 passos $\times$ 1,5).

Tabela 5. Resultado de swing-up por n (linha principal, observação completa, exceto onde indicado). %up = fração do tempo com *todos* os elos em $\cos \theta > 0,6$, medida a posteriori sobre o checkpoint final (1024 episódios de avaliação).

Config	Política	Retorno / teto	%up	Veredito
$n = 1$ full-obs	MLP	2188 / 2250	97%	Resolvido
$n = 2$ full-obs (50B)	MLP	2121 / 2250	95,5%	Resolvido
$n = 3$ full-obs (50B)	MLP	1930 / 2250	88,4%	Quase resolvido
$n = 1$ POMDP $(x, \dot{x})$	LSTM	—	96,8% (245B)	Resolvido às cegas
$n = 1$ POMDP $(x, \dot{x})$	MLP	—	$\sim 11\%$	Controle negativo: falha
$n = 2$ POMDP	LSTM	—	$\sim 38\%$	Em progresso
$n = 4$ full-obs	MLP	1031 / 2250	56,4%	Parcial (500B)
$n = 5$ full-obs	MLP	610–624 / 2250 (27%)	39,9%	Não resolvido (1,6T)

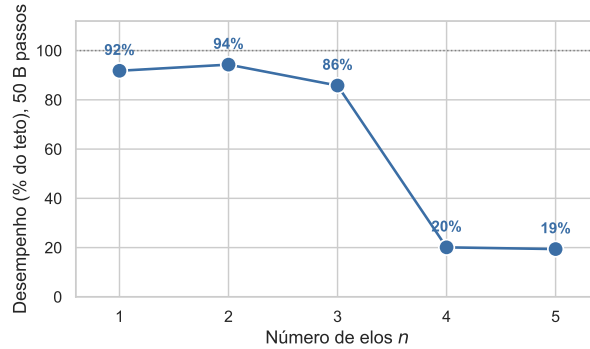


Figura 3. Desempenho do swing-up (% do teto de recompensa) em função do número de elos n , sob o mesmo orçamento de 50 B passos e o mesmo código para todos os n : o “clique” do swing-up ocorre dentro do orçamento até $n=3$; para $n \geq 4$ ele não ocorre em 50 B — e, no caso de $n=5$, nem com $32\times$ mais orçamento (Figura 4). Corridas de semente única; a diferença entre $n=1$ e $n=2$ está dentro do ruído entre sementes.

Resultado em $n = 1$. Com observação completa, o swing-up é resolvido com folga (2188 de 2250; 97% do tempo ereto). Mais notável é a variante às cegas: observando *apenas* o carrinho, a LSTM com TBPTT stateful atinge 96,8% ereto aos 245 bilhões de passos — estatisticamente o mesmo teto do agente que enxerga tudo. A memória recupera integralmente a observabilidade; o custo da cegueira é eficiência amostral (~ 20 – $40\times$ mais passos), não teto de desempenho. O controle negativo confirma o desenho: a mesma observação parcial com MLP sem memória estaciona em $\sim 11\%$.

Resultado em $n = 5$. A Figura 4 mostra as curvas de aprendizado sob o mesmo orçamento: o $n=5$ nunca sai do platô de sobrevivência dentro dos 50 B. Estendendo o treino muito além, a progressão é real, mas brutalmente lenta: retorno 61 aos 100 milhões de passos, 372 aos 500 milhões, platô em ~ 470 por centenas de bilhões, e *warm-restarts* de LR arrancando ganhos decrescentes — 530 $\rightarrow$ 612,6 no primeiro ciclo (1T), sem recuperação do nível anterior no segundo (2T, interrompido a 1,4T por reboot; retomou a ~ 575). O melhor resultado, ~ 610 – 624 de 2250 ($\sim 27\%$ do teto), foi obtido com 1,6 trilhão de passos acumulados. Os episódios sobrevivem os 1500 passos inteiros: a política aprendeu a não falhar, mas não a completar o swing-up dos 5 elos. É um platô de otimização/exploração, não de sobrevivência.

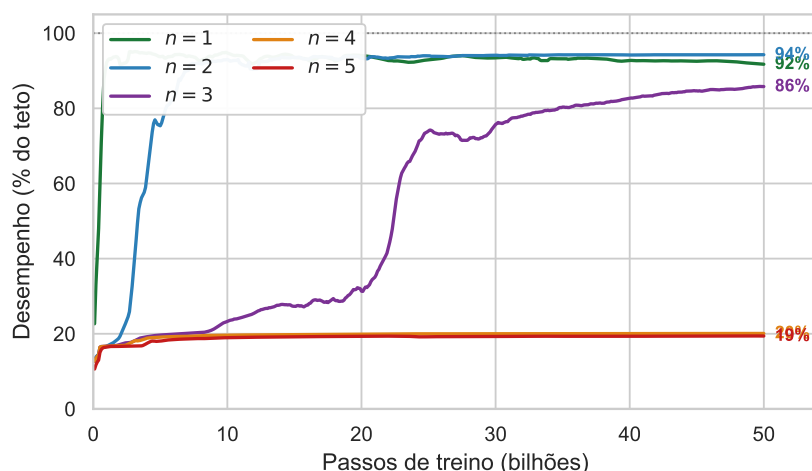


Figura 4. Curvas de aprendizado sob o mesmo orçamento de 50 B passos, $n = 1$ a 5 (recompensa de avaliação, % do teto de 2250). Até $n=3$ o “clique” do swing-up ocorre e o desempenho converge acima de 85%; $n=4$ e $n=5$ permanecem no platô de sobrevivência ($\sim 20\%$). O $n=5$ foi treinado adicionalmente até 1,6 T passos e alcançou apenas $\sim 27\%$ (ver texto).

5. Análise

5.1. Pilar 1 — Por que throughput é a alavanca: capacidade não é o gargalo

O argumento central se sustenta em três evidências convergentes. Primeiro, a dimensionalidade: a observação de $n=5$ tem 17 números; a de $n=21$ teria 65. Uma MLP 2×256 ($\sim 140K$ parâmetros) é vasta para funções nesse domínio — e de fato o $n=1$ é resolvido com folga por essa rede, inclusive na versão POMDP em que a LSTM precisa manter um filtro de estado interno. Segundo, o modo de falha do $n=5$: os episódios não terminam por queda — a política domina a sobrevivência — mas o retorno estaciona. Se capacidade fosse o gargalo, veríamos subajuste também nas competências parciais; o que vemos é uma política competente presa em uma bacia de comportamento, sintoma de *exploração* insuficiente sobre uma paisagem de recompensa com gradiente quase nulo entre “balançar alto” e “completar o swing-up de 5 elos”. Terceiro, a aritmética do orçamento: foi o bf16 ($1,87\times$), e não uma mudança algorítmica, que transformou semanas de GPU em dias e tornou viável o experimento de 1,6 trilhão de passos. Nesse regime, otimizações de sistemas determinam quais perguntas de pesquisa cabem no hardware disponível.

5.2. Pilar 2 — Precisão vs. kernelização: lições de medição

Baseline de produção, não espantinho. CUDA graphs pareceram render $1,52\times$ — mas contra uma réplica com política *aleatória*, que derruba o pêndulo e gera ~ 42 mil sincronizações device→host por época. Contra a produção real (política treinada, que completa os 1500 passos e quase não sincroniza) o ganho é $\sim 1,05\times$: paridade. O mesmo condenou o Triton ($2,88\times$ micro, $+2\%$ real), cujo microbenchmark comparava contra um cuBLAS artificialmente lento. Ou seja: um speedup medido contra uma *baseline* fraca não se sustenta após a integração no sistema real.

Throughput não é o objetivo, aprendizado é. Reusar gradientes por 2 minibatches deu $1,35\times$ e *pareceu* inócuo por 500M passos; sob observação sustentada (2,5B) o retorno degradou monotonicamente ($454 \rightarrow 426$) e foi revertido. Um A/B curto pode aprovar uma perda lenta.

Medir o valor físico, não o proxy. No *center-farming* (Seção 3.4), o controle negativo (MLP cega) teve retorno *maior* que o full-obs (212 vs. 66) — parado no centro, colhendo o termo center com o pêndulo pendurado; só a métrica física ($\cos \theta$ do simulador) revelou o hack.

5.3. Pilar 3 — Onde o PPO quebra ao crescer n

Sob orçamento uniforme, a transição não é gradual: de $n=3$ (86%) para $n=4$ (20%) o desempenho *despenca* (Figura 3). O “clique” do swing-up, que ocorre em menos de 1 B de passos para $n=1$, ~ 5 B para $n=2$ e ~ 20 B para $n=3$ (Figura 4), simplesmente não acontece para $n \geq 4$ nesse orçamento. A causa é identificável. O espaço de estados cresce ($2n + 2$ dimensões), mas o fator dominante é a combinação de **subatuação e caos**: com um único atuador para $n+1$ graus de liberdade, achar a sequência de chaveamentos que faz todos os elos ganharem energia de forma coerente fica exponencialmente mais difícil, e a dinâmica caótica pune qualquer erro de fase. A recompensa densa em altura (Eq. 3) guia os primeiros elos, mas o gradiente entre “4 elos oscilando alto” e “5 elos no topo” é quase invisível para a exploração. Os warm-restarts de LR extraíram $+15\%$ com retornos decrescentes, o esperado quando o gargalo é a exploração, não a otimização local. As saídas prováveis (currículo sobre n ou gravidade, transferência $n-1 \rightarrow n$, *shaping* de energia [Åström e Furuta 2000]) mudam a paisagem, não o otimizador.

5.4. Pilar 3 — O resultado POMDP e o mecanismo do TBPTT

O experimento às cegas quase falhou por um motivo específico: como o BPTT reiniciava o estado oculto em $h=0$ a cada janela de 128 passos, a LSTM era treinada sobre um estado inicial diferente daquele que de fato gerou as ações, acumulado ao longo de até 1500 passos. Essa inconsistência corrompia a atribuição de crédito. Com esse defeito, a recorrente empatava com o controle negativo ($\sim 11\%$). O TBPTT stateful (janela inicializada com o estado oculto real do rollout) destravou a curva: 25% aos 9B, 50% aos 40B, 96,8% aos 245B. O resultado final mostra que é possível inferir o estado de um pêndulo nunca observado diretamente, apenas a partir das reações do carrinho, e ainda assim invertê-lo. Entre os resultados deste artigo, é o que melhor ilustra que o PPO, com orçamento de amostras suficiente, resolve problemas não triviais de estimação e controle conjuntos.

5.5. Limitações

Com a mesma honestidade: (i) $n=4$ é parcial e $n=5$ está *não resolvido* — 27% do teto após 1,6T passos; throughput sozinho não venceu a paisagem de exploração para $n \geq 4$. (ii) A linha principal em CUDA ainda carrega o termo center farmável na recompensa (Eq. 3); a correção existe apenas na variante POMDP, e os retornos de $n \geq 4$ devem ser lidos com essa ressalva. (iii) A métrica %up não foi registrada durante os treinos; foi medida a posteriori sobre os checkpoints finais (Tabela 5). (iv) Os resultados de throughput são de uma única GPU (RTX 3060); as conclusões roofline devem transferir para outras arquiteturas memory-bound, mas não foram verificadas nelas.

6. Conclusões

Este trabalho tratou o swing-up do pêndulo- N subatuado como um problema de *orçamento de amostras*, estudado nos três eixos do título. Em **throughput**, um ambiente pêndulo- N genérico e 100% residente em GPU sustenta 7–8M passos/s ponta a ponta em uma RTX 3060 — a alavanca que financia a exploração, já que a capacidade da rede não é o gargalo. Em **precisão mista**, o resultado central é que, num laço memory-bound, a *precisão* (bf16, $1,87\times$ sem degradar o aprendizado) vale mais que a kernelização (que rendeu paridade) — e a própria metodologia de medição (baseline de produção, A/B multi-semente sustentado) é contribuição. Em **escalabilidade**, o sistema escala (trilhões de passos em GPU de consumo) e é isso que permite *tentar* escalar o problema: sob 50 B de passos o PPO resolve até $n=3$ (94% e 86% do teto; em $n=1$ inclusive às cegas por LSTM, com 96,8% do tempo ereto), despenca em $n=4$ (20% em 50 B; 46% mesmo com 500 B) e para em $n=5$ (27% após 1,6 T). Escalar $n \geq 4$ permanece, explicitamente, fronteira aberta.

Trabalhos futuros, em ordem de prioridade: (i) atacar $n \geq 5$ mudando a paisagem de exploração (currículo sobre n ou gravidade, transferência $n-1 \rightarrow n$, shaping de energia [Åström e Furuta 2000]); (ii) vieses estruturais para generalizar entre n (codificador por elo, 1D-conv, message-passing), pré-requisito para (iii) subir a escada até $n=21$; e (iv) concluir o $n=2$ às cegas, hoje em $\sim 38\%$.

Referências

- Åström, K. J. e Furuta, K. (2000). Swinging up a pendulum by energy control. *Automatica*, 36(2):287–295.
- Barth, F. (2026). Proximal Policy Optimization (PPO). Notas de aula, Insper. https://insper.github.io/rl/classes/18_ppo/.
- Boubaker, O. (2013). The inverted pendulum benchmark in nonlinear control theory: a survey. *International Journal of Advanced Robotic Systems*, 10(5):233.
- Freeman, C. D., Frey, E., Raichuk, A., et al. (2021). Brax — a differentiable physics engine for large scale rigid body simulation. Em *NeurIPS Datasets and Benchmarks Track*.
- Makoviychuk, V., Wawrzyniak, L., Guo, Y., et al. (2021). Isaac Gym: High performance GPU-based physics simulation for robot learning. Em *NeurIPS Datasets and Benchmarks Track*.
- Micikevicius, P., Narang, S., Alben, J., et al. (2018). Mixed precision training. Em *International Conference on Learning Representations (ICLR)*.
- Petrenko, A., Huang, Z., Kumar, T., Sukhatme, G., e Koltun, V. (2020). Sample Factory: Egocentric 3D control from pixels at 100000 FPS with asynchronous reinforcement learning. Em *International Conference on Machine Learning (ICML)*.
- Schulman, J., Moritz, P., Levine, S., et al. (2016). High-dimensional continuous control using generalized advantage estimation. Em *International Conference on Learning Representations (ICLR)*.
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., e Klimov, O. (2017). Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*.
- Shacklett, B., Rosenzweig, L. G., Xie, Z., Sarukkai, B., e Fatahalian, K. (2023). An extensible, data-oriented architecture for high-performance, many-world simulation. *ACM Transactions on Graphics*, 42(4).
- Spong, M. W. (1995). The swing up control problem for the Acrobot. *IEEE Control Systems Magazine*, 15(1):49–55.
- Suárez, J. (2024). PufferLib: Making reinforcement learning libraries and environments play nice. *arXiv preprint arXiv:2406.12905*.
- Weng, J., Lin, M., Huang, S., et al. (2022). EnvPool: A highly parallel reinforcement learning environment execution engine. Em *Advances in Neural Information Processing Systems (NeurIPS)*.